Prompt 1

First, inspect my existing project structure, Kotlin or Java version, Gradle configuration, Android SDK settings, manifest, dependencies, and existing layouts.

My assignment requires:

1. A Navigation Drawer with Broadcast Receiver, Image Scale, Video, and Audio menu options.
2. A three-stage broadcast workflow, including a custom text broadcast and a battery percentage receiver.
3. An online image that supports pinch-to-zoom.
4. In-app video playback.
5. In-app audio playback.

Explain my existing project structure in beginner-friendly language. Identify what is already implemented and what is missing.

Do not rewrite the whole project or change working files unnecessarily. Before making changes, give me a step-by-step implementation plan. Use Kotlin and XML layouts if compatible with my project; if the project uses a different approach, explain that before changing it.

Prompt 2

Requirements:

- Create a Navigation Drawer with these four menu items in this order: Broadcast Receiver, Image Scale, Video, Audio.
- Add a toolbar with a hamburger menu icon.
- Show an appropriate title for the selected menu item.
- Use Android Jetpack components and a clean, maintainable structure.
- Use a NavigationView and DrawerLayout if appropriate for my XML-based project.
- Use fragments for the main drawer destinations where appropriate.
- Keep the broadcast workflow's required activities separate.
- Make each menu item navigate to its correct screen.
- Provide a home screen or sensible initial destination.
- Add all required layouts, menu XML files, manifest declarations, and Gradle dependencies.

Preserve my existing project configuration where possible. Give me the exact file paths for all created or modified files. Explain the code in simple terms and ensure the project compiles before proceeding.

Prompt 3 — Implement the three-stage broadcast flow

Implement the complete Broadcast Receiver feature according to my assignment PDF.

Activity 1:

- Show a Spinner with exactly two options:
    1. Custom Broadcast Receiver
    2. System Battery Notification Receiver
- Add a Proceed button.
- Navigate according to the selected option.

Custom option:

- Activity 2 contains an EditText for entering plain text and a button to continue.
- Pass the entered text to Activity 3 using an Intent extra.
- Activity 3 registers or creates a BroadcastReceiver for a custom application broadcast action.
- When Activity 3 is ready, send an explicit custom broadcast containing the text.
- Receive the broadcast and display the exact message in Activity 3.
- Avoid sending the broadcast before the receiver is registered.
- Validate empty input and provide a useful message.

Battery option:

- Activity 2 obtains and displays the actual battery percentage from the Android system.
- Use ACTION_BATTERY_CHANGED appropriately, understanding that it is a sticky broadcast and is not normally registered using the same pattern as ordinary broadcasts.
- Update the percentage when appropriate.
- If the user proceeds to Activity 3, do not perform the custom-message operation for this option.

Use correct receiver registration and unregistration, manifest declarations where required, and Android API-level-compatible exported/not-exported flags. Do not request unnecessary permissions. Explain the difference between an Intent extra and a broadcast.

Provide every required Kotlin file and XML layout, exact file paths, and testing steps. Keep the implementation consistent with my existing project.

Prompt 4 — Image scaling

Implement the Image Scale feature in my existing Android application.

Requirements:

- Display an image loaded from a valid internet image URL.
- Load the image asynchronously using a suitable image-loading library such as Coil or Glide, compatible with my project.
- Add pinch-to-zoom using a two-finger gesture.
- Allow the user to zoom in and out smoothly.
- Keep the image within the screen and handle different screen sizes.
- Show a loading indicator and a useful error message if the image fails to load.
- Add the INTERNET permission if necessary.
- Integrate the feature into the existing Image Scale drawer destination.

Use Kotlin and the project's existing UI approach. Do not fake zooming by simply changing the image dimensions without implementing a real pinch gesture.

Provide all required files, dependencies, and exact file paths. Explain how to test the gesture on an emulator or physical device.

Prompt 5 — Video player

Implement the Video feature in my existing Android application for CSE 489 Assignment 2.

Requirements:

- Play one video within the app.
- Use Android VideoView or AndroidX Media3, choosing the simplest reliable option compatible with my project.
- Provide Play, Pause, and playback progress controls where supported.
- Use a local video file stored in app/src/main/res/raw/ so playback does not depend on an external website.
- Explain the required filename format and how I should add the video asset.
- Handle missing or invalid media gracefully.
- Integrate the player into the Video drawer destination.

- Release or manage playback resources appropriately when the screen is closed.

Provide exact file paths, any Gradle dependencies, and all necessary Kotlin and XML code. Do not invent a media file that is not actually present in my project; tell me if I need to add one.

Prompt 6 — Audio player

Implement the Audio feature in my existing Android application for CSE 489 Assignment 2.

Requirements:

- Play one audio file inside the app.
- Use Android MediaPlayer or another suitable simple Android playback API compatible with my project.
- Store the audio file in app/src/main/res/raw/.
- Provide Play, Pause, and Stop controls, plus playback progress if practical.
- Explain how to add and name the audio asset.
- Handle missing or invalid audio files.
- Release MediaPlayer resources correctly and prevent resource leaks.
- Integrate the player into the Audio drawer destination.
- Do not require internet access to play the local audio file.

Provide every required file, exact file paths, and simple instructions for testing.